

Phase Shifter Interface

Standardy Komunikacji Międzyukładowej w modułowych systemach wbudowanych

Kamil Chaj

1 Introduction

This project is a part of the phased array antenna and the goal is to create a way to programmable control multiple phase shifters. Phase shifters can be controlled directly by accessing all control bits in parallel, or they can be controlled via the SPI-like protocol, which enables controlling multiple phase shifters utilizing minimal pin count. The interface with the microcontroller will be implemented using UART.

2 Selected MCU

The STM32 NUCLEO-G0B1RE evaluation board was chosen for this project, the main requirement was an integrated USB controller to minimize the element count on the future printed circuit board. Also, the G0 series MCU was chosen because of low price and no need for high performance of specific peripherals other than USB and SPI.

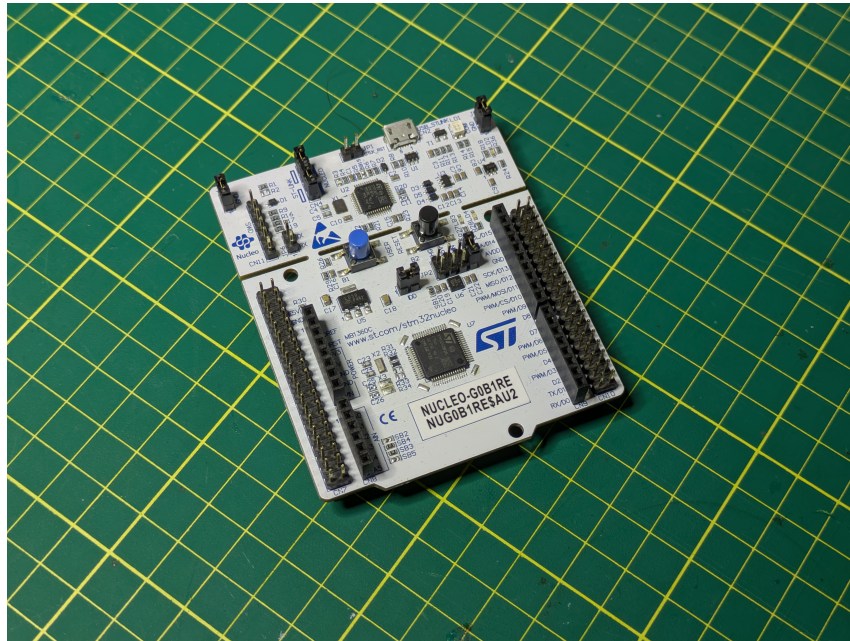


Figure 1: NUCLEO development board

3 SPI protocol

[MAPS-010144](#) phase shifter has a 6-bit shift register which corresponds to quantized phase shift, in the case of this particular phase shifter 2 least significant bits are not used because the maximum resolution is 4-bit. In order to update the phase shift from the register to the actual phase shift, the LE pin must be set. Phase shifters are equipped in both SERIN and SEROUT pins which allow multiple ICs to be connected in a daisy chain.

SPI peripheral was configured according to the phase shifter data sheet, data size was set at 6 bits and the transmission starts from the least significant bit; also, the baud rate was set to just 250 Kbits per second, which is much slower than what the IC can handle but what allowed easier debugging with my current measurement equipment.

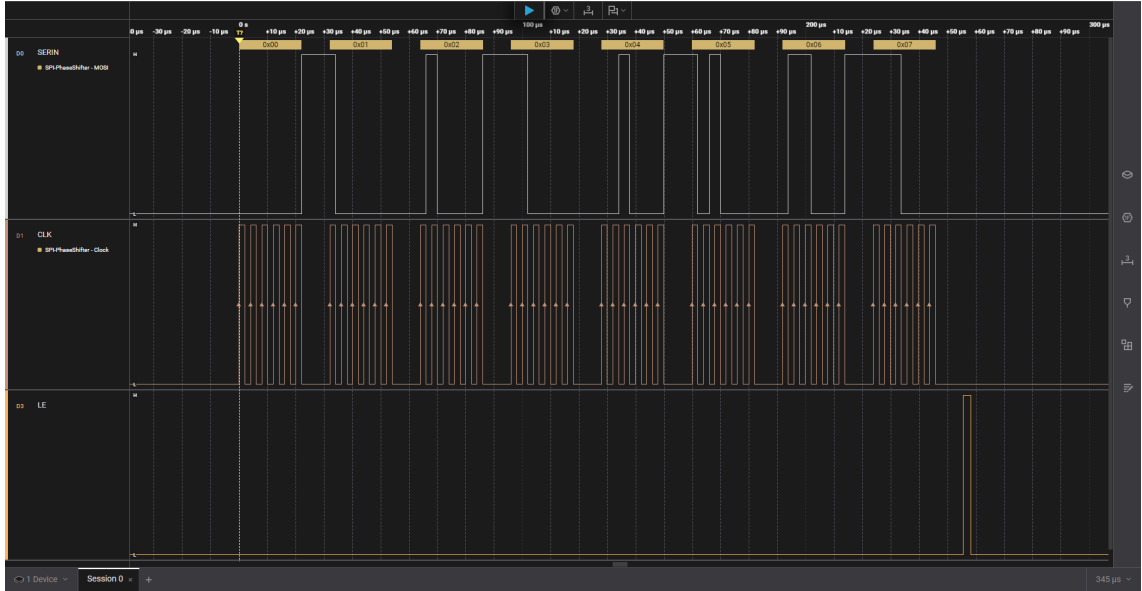


Figure 2: SPI communication captured with logic analyzer

4 USB CDC virtual COM port

Communication over virtual COM port is implemented using the USB_DEVICE middleware provided with the STM32CUBE environment, the middleware is set in the Communication Device Class (CDC) mode. Using this middleware allows for simple communication with a PC identical to UART.

4.1 Communication protocol

Byte 0	Byte 1	Byte 2	Byte...	Byte N	Byte N+1	Byte N+2
COMMAND ID	DATA SIZE	DATA 0	...	DATA N	CRC	END

Table 1: Data format

Name	Hex value	Description
OK	0x00	transmission successful
ECHO	0x01	return the same command
ERROR	0xff	transmission failed
PHASE SHIFTER COUNT	0x03	set number of phase shifters
SEND BUFFER	0x10	begin SPI transmission
SET BUFFER	0x11	send new buffer
GET BUFFER	0x12	request current buffer
GET BUFFER RE	0x13	response with buffer
CLEAR BUFFER	0x14	set buffer to 0

Table 2: Commands

4.2 Cyclic redundancy check

CRC value on MCU side is calculated using integrated CRC calculation unit and the same algorithm is implement on PC side.

5 Implementation

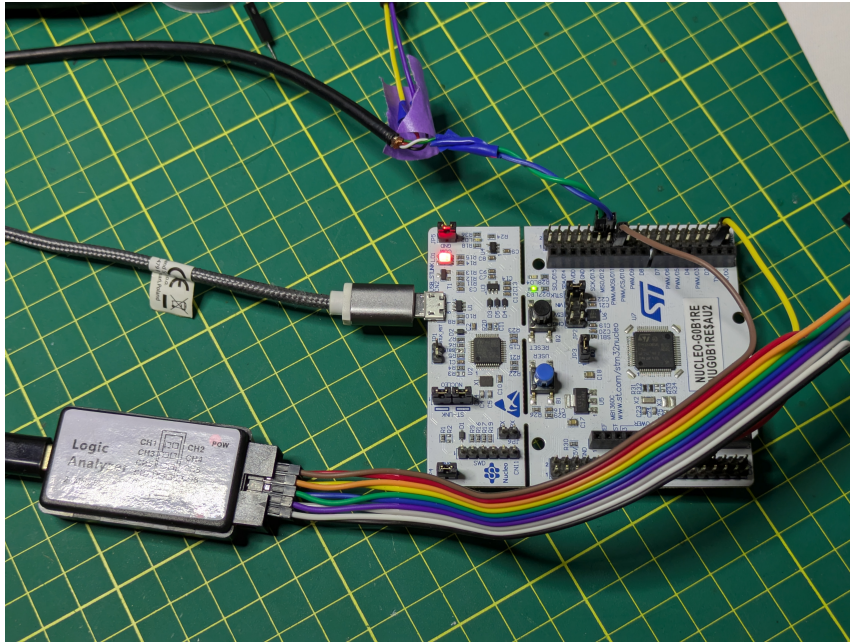


Figure 3: Test setup

5.1 Peripheral initialization

The first step of implementation was setting up the required peripherals in STM32CubeMX:

- TIM1 - timing LE pulse
- SPI1 - communication with phase shifter
- USART2 - ST-LINK virtual com port for debugging
- USB_DRD_FS - integrated USB peripheral
- CRC - CRC calculation unit
- USB_DEVICE - Middleware for USB

5.2 Phase shifter interface

As for typical embedded program structure, I create a **typedef struct** `phase_shifter` which holds all necessary handles and variables, and **typedef enum** `phase_shifter_status` for status type.

after defining those types, I created basic functions that implement the following actions: initiation of `phase_shifter`, setting, getting and clearing buffer, setting the number of connected phase shifters and starting transmission.

5.3 PC-MCU communication

Similarly as in the previous section, I defined **typedef struct** `command`, **typedef struct** `interface` and **typedef enum** `interface_status`. The command struct is used to store and create commands in a more convenient way during processing.

Functions used for communication:

- `recieve_command` - dissects buffer and populates variables in command struct, verifies CRC value
- `transmit_command` - assembles buffer from command struct and appends CRC value
- `process_command` - handles received commands and sends responses to the sender

The entire communication is PC-driven, which means that MCU does something in response to the PC.

5.4 Python API

As mentioned in the introduction, the goal is to create a way to programmable control phase shifter therefore, I created a simple Python program that allows for sending and receiving data.

5.5 Block diagram

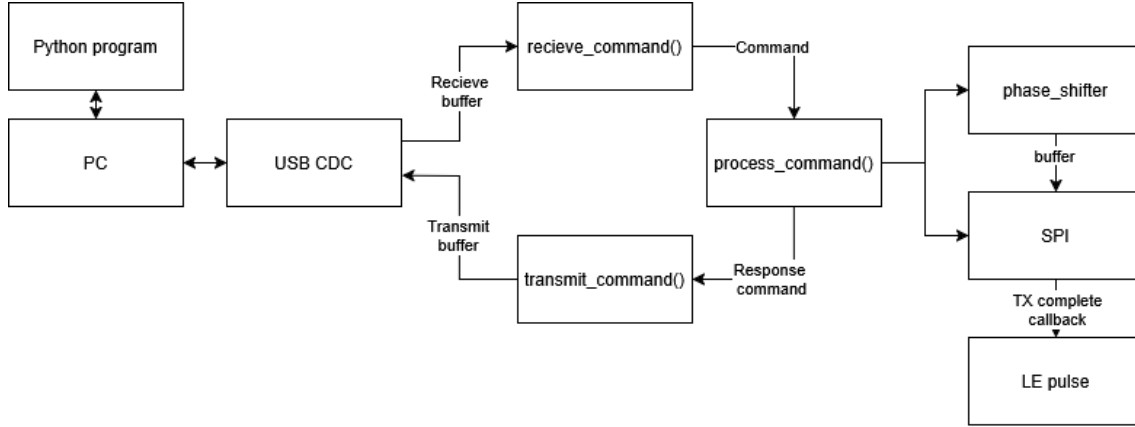


Figure 4: Block diagram of the program

6 Conclusions

This project successfully demonstrates a complete system for controlling multiple integrated circuits using embedded communication protocols. By integrating USB CDC communication, a structured command protocol, SPI data transmission, and a Python-based control API, a flexible and scalable interface was established. The modular software structure, including clear abstractions for the phase shifter interface and communication protocol, enhances maintainability and potential for future expansion.

Overall, the project fulfills its intended goals and provides a solid foundation for further development and integration into more complex RF systems.

All code available on [GitHub](#)